RE·TECH FUSION HACKATHON · INSAT, UNIVERSITY OF CARTHAGE

NRTF Part 1 — ESP32 IoT Sensor Device

A modular, MQTT-based telemetry firmware for the ESP32 WROOM-32D with offline buffering, connection-loss recovery, and data quality tracking.

Repository: https://github.com/devaminebk/NRTF · Path: `part1/` · Board: `esp32doit-devkit-v1` · Framework: PlatformIO + Arduino

1. What this firmware does

Part 1 of the NRTF project turns an ESP32 WROOM-32D into a self-contained IoT data node that reads sensors, validates the readings, and publishes them over MQTT to a local Mosquitto broker as structured JSON. The architecture is designed for the hackathon scoring rubric — every category (multi-sensor coordination, protocol design, uptime, data quality, innovation) is addressed by an explicit subsystem rather than ad-hoc code.

Sensors integrated

- **DHT22** — temperature (°C) and humidity (%)
- **Flame sensor** — digital trigger + analog intensity (raw 0–4095)

Headline features

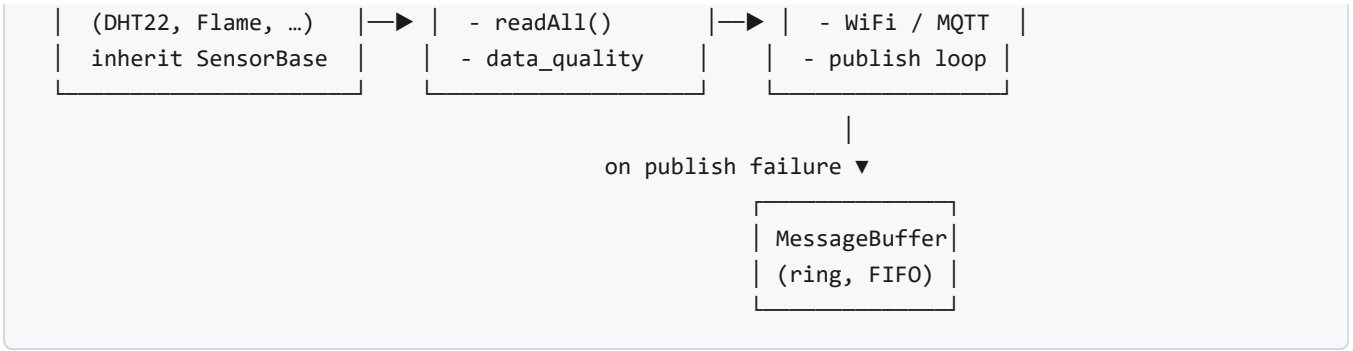
- Plugin-style sensor architecture
- WiFi + MQTT auto-reconnect
- Offline ring buffer with FIFO replay
- Cumulative data-quality percentage
- Per-measurement units in the JSON payload

2. Hardware wiring

Component	ESP32 Pin	Notes
DHT22 Data	GPIO 4	10 kΩ pull-up to 3.3 V required
Flame Digital	GPIO 5	1 = flame detected, 0 = no flame
Flame Analog	GPIO 34	ADC1 input-only pin (0–4095)
Common GND	GND	Shared between all sensors
Power	3.3 V / 5 V	Per sensor datasheet

3. Software architecture

The firmware uses a plugin-style sensor architecture so additional sensors can be added without modifying any existing file. Three layers cooperate:



Adding a new sensor

1. Create `include/sensors/your_sensor.h` inheriting `SensorBase`.
2. Create `src/sensors/your_sensor.cpp` implementing `init()`, `read(JsonObject&)`, `getName()`.
3. Register in `setup()`: `sensorManager.addSensor(new YourSensor(pin));`

The sensor's data automatically appears in every published payload and contributes to the cumulative data-quality counter.

4. MQTT data format

Published every 5 seconds to topic `esp32/sensors`:

```
{
  "timestamp": 12345678,
  "sensors": {
    "DHT22": {
      "temperature": { "value": 25.5, "unit": "C" },
      "humidity":    { "value": 60.2, "unit": "%" }
    },
    "FlameSensor": {
      "flame_digital": { "value": 0,    "unit": "bool" },
      "flame_analog":  { "value": 1024, "unit": "raw"  }
    }
  },
  "data_quality": {
    "valid_reads": 47,
    "total_reads": 48,
    "valid_pct":   97.9
  }
}
```

> **Why nested value/unit objects?** A downstream consumer can render a unit-aware UI without hard-coding sensor knowledge, and adding units after the fact would have been a breaking change for any subscriber.

5. Connection-loss recovery

A single MQTT broker is a single point of failure. The firmware therefore treats network and broker availability as ephemeral and degrades gracefully when either disappears. Detection is layered for speed:

Layer	Catches	Time to detect
WiFi event handler (`WiFi.onEvent`)	AP loss, hotspot turned off	~6 s (driver beacon timeout)
TCP socket (`WiFiClient.connected`)	Broker process killed (FIN/RST)	1–3 s
MQTT keepalive (PINGREQ timeout)	Half-open socket, NAT timeout	~8 s (`2 × MQTT_KEEPALIVE`)

Offline buffering & FIFO replay

When any of those checks indicate the link is down, the publish path skips `mqttClient.publish()` entirely (which would otherwise return a false success by writing to a dead lwIP send buffer) and pushes the serialized payload — together with the original capture timestamp — into a fixed-size circular buffer in RAM (`BUFFER_MAX_SIZE = 20` entries by default).

When the buffer fills, the oldest entry is discarded so the most recent data is always preserved. The instant a new MQTT connection is established, `flushBuffer()` is invoked from the reconnection path and drains the queue in FIFO order, logging each replayed message under the `[Backup]` tag so they can be visually distinguished from live publishes in the serial monitor.

6. Data quality tracking

Each individual sensor read is counted as either valid (returned `true`) or invalid (NaN, out-of-range, or a sentinel like −999). The `SensorManager` maintains cumulative counters and emits them in every payload so downstream services can monitor sensor health over time.

Validation rule	Where defined
Temperature ∈ [−40, 80] °C	`config.h · TEMP_MIN/MAX`
Humidity ∈ [0, 100] %	`config.h · HUMIDITY_MIN/MAX`
Flame analog ∈ [0, 4095]	`config.h · FLAME_ANALOG_MIN/MAX`
NaN rejection on every float read	each sensor's `read()`

7. Configuration (`include/config.h`)

```
// WiFi / MQTT
#define WIFI_SSID        "Galaxy S20 Fe"
#define WIFI_PASSWORD    "••••••••"
#define MQTT_SERVER      "192.168.20.15"
#define MQTT_PORT        1883
#define MQTT_TOPIC_SENSORS "esp32/sensors"

// Pins
#define DHT22_PIN          4
#define FLAME_DIGITAL_PIN  5
#define FLAME_ANALOG_PIN  34

// Cadence + liveness
#define SENSOR_READ_INTERVAL    5000   // ms
#define MQTT_RECONNECT_INTERVAL 5000   // ms
```

```
#define MQTT_KEEPALIVE             4   // s — aggressive PINGREQ
#define MQTT_SOCKET_TIMEOUT        2   // s — give up fast on dead socket

// Offline buffer
#define BUFFER_MAX_SIZE   20
#define PAYLOAD_MAX_LEN  512
```

8. Build, upload, monitor

From inside `part1/` :

```
pio run                    # compile
pio run --target upload    # flash to ESP32 (COM5 by default)
pio device monitor         # serial @ 115200
```

> **Workspace tip:** open `NRTF.code-workspace` at the repo root rather than the `NRTF/` folder directly, otherwise PlatformIO will not detect `part1/` as a project and IntelliSense will fail to resolve includes.

9. Scoring rubric alignment

Criterion	Implementation
Solution delivered & functional (30 pts)	Validated JSON published every 5 s to MQTT
Multi-sensor coordination (25 pts)	DHT22 + Flame read in a single cycle via `SensorManager`
Protocol design & security (15 pts)	MQTT with keepalive + auto-reconnect (TLS-ready)
Uptime & continuity (15 pts)	Event-driven WiFi reconnection, offline ring buffer with FIFO replay
Data quality (10 pts)	Range & NaN validation, cumulative `valid_pct` in every payload
Innovation bonus (15 pts)	Plugin sensor architecture, layered connection detection, units in payload